

FinOps HANDBOOK

A Practical Guide for Junior Engineers

Understand Cloud Costs | Build Cost-Aware Systems
Optimize Resources | Make Smarter Engineering Decisions



20
PAGES



REAL
EXAMPLES



PRACTICAL
EXERCISES



PRODUCTION
FOCUS

CLOUD COSTS
REAL KNOWLEDGE
REAL CAREERS

VERIQTA
ENGINEER TODAY. A BRIGHTER TOMORROW.

SAME
DISCIPLINE
LOWER COSTS
BIGGER IMPACT

FinOps for Engineers

WHERE CLOUD MONEY ACTUALLY GOES

Understand Costs | Build Smarter Systems | Create Real Business Value

1 WHAT IS FINOPS?



- FinOps (Financial Operations) is a practice that helps organizations understand, manage, and optimize cloud costs.
- It brings together engineering, finance, and business teams to get more value from cloud spending.

2 WHY CLOUD COST IS AN ENGINEERING CONCERN



- Every infrastructure decision has a cost impact.
- Engineers choose instance types, storage, regions, and services.
- Small decisions at scale can create large costs.
- Cost-aware engineers build more efficient and sustainable systems.

3 CLOUD SPENDING vs TRADITIONAL INFRASTRUCTURE



- Traditional infrastructure requires large upfront investment.
- Cloud is pay-as-you-go with no large hardware purchase.
- Cloud provides flexibility, but costs can grow quickly if not managed.

4 PAY-AS-YOU-GO AND VARIABLE CLOUD COSTS



- You only pay for what you use.
- Costs can increase or decrease based on usage.
- Costs are variable, not fixed.
- It's easy to scale up, but costs can also scale up unexpectedly.

5 COST, USAGE, AND BUSINESS VALUE



- Cost is the amount you pay.
- Usage is how much you consume (compute, storage, data, etc.).
- Business value is the benefit the application or service provides.
- FinOps helps you increase business value for every dollar spent.

6 WHY "REDUCE THE BILL" IS NOT THE FULL MEANING OF FINOPS



- FinOps is not just about cutting costs.
- It's about getting more value from cloud spending.
- The goal is to spend smarter, not just spend less.
- Sometimes you need to spend more to deliver a better, more reliable service that creates business value.

7 WHO IS INVOLVED?



- Engineering:** builds and operates the infrastructure.
- Finance:** tracks spend and provides financial guidance.
- Procurement:** manages contracts and vendor agreements.
- Business:** sets goals and evaluates business value.
- FinOps is a shared responsibility, not just one team's job.

8 THE FINOPS LIFECYCLE

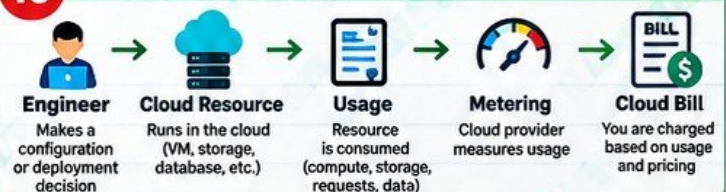


9 DEVOPS CONNECTION



- Every DevOps and infrastructure decision has a cost consequence.
- Good architecture, automation, and observability help control costs.
- Cost-aware engineers build reliable, efficient, and scalable systems.

10 HOW CLOUD MONEY FLOWS



Junior Engineer Tip:

Always ask: What will this cost? What value does it create?

Think in value, not just cost.



@veriqta



veriqta



@veriqta



@veriqta



@veriqta



@veriqta

How Cloud Providers Charge You

DIFFERENT SERVICES. DIFFERENT PRICING MODELS. SAME BILL.

Understand how cloud providers charge so you can make better engineering decisions.

1

COMPUTE PRICING



- You pay for instance type (CPU, memory).
- Billed per second or per hour.
- Costs vary by size, region and OS (Linux is cheaper).
- Stops when you stop the instance.

2

STORAGE PRICING



- You pay for the amount of data stored (GB/TB).
- Different storage classes (standard, infrequent access, archive).
- You may also pay for requests (read/write).
- Keeping old or unused data increases costs.

3

DATABASE PRICING



- You pay for instance size, storage, and IOPS.
- Managed databases cost more than self-managed.
- Backups, replicas and Multi-AZ increase costs.
- Costs vary by engine (e.g. PostgreSQL, MySQL, Oracle, SQL Server).

4

NETWORK AND DATA-TRANSFER CHARGES



- You pay for data moving in and out of the cloud (egress).
- Internal traffic (between regions or availability zones) can also cost.
- Services like NAT Gateway, Load Balancers and CDN add network costs.

5

REQUEST-BASED PRICING



- You pay per API request or transaction.
- Common in services like object storage, databases, and serverless.
- High request volumes can lead to high costs.
- Example: S3 GET/PUT requests, DynamoDB reads and writes.

6

MANAGED-SERVICE CHARGES



- Managed services (e.g. RDS, EKS, ElastiCache) include operational convenience.
- You pay a premium for automation, high availability and management.
- Less operational work, but higher cost than self-managing.

7

SERVERLESS EXECUTION COSTS



- You pay for the number of requests and execution time.
- Billed per GB-second.
- Costs scale automatically with usage.
- Can be very cost-effective, but can become expensive with high or long-running workloads.

8

KUBERNETES COSTS



- You pay for the underlying compute (worker nodes).
- Additional costs for load balancers, storage and network.
- Unused or overprovisioned nodes increase costs.
- Kubernetes itself is free, but the cloud resources it uses are not.

9

LICENSING COSTS



- Some software requires licenses (e.g. Windows Server, SQL Server, Oracle).
- Bring Your Own License (BYOL) or provider licenses can affect pricing.
- Always check licensing costs before deployment.

10

REGIONAL PRICING DIFFERENCES



- Prices vary by region (e.g. us-east-1 vs eu-west-1).
- Some regions are more expensive due to demand, infrastructure and local taxes.
- Data transfer between regions also costs money.

11

ON-DEMAND vs COMMITMENT-BASED PRICING



- On-demand: pay as you go, no long-term commitment.
- Commitment-based: lower prices in exchange for commitment (e.g. Reserved Instances, Savings Plans).
- Best for predictable workloads.

12

HIDDEN COST DRIVERS BEGINNERS OVERLOOK



- Forgotten or idle resources.
- Unattached storage volumes.
- Large log retention.
- Data transfer charges.
- Snapshots and backups.
- NAT gateways and load balancers.
- Unnecessary high availability in non-production.
- Unoptimized resource sizing.

13

PRACTICAL EXERCISE

IDENTIFY THE COST DIMENSIONS BEHIND A SIMPLE WEB APPLICATION



Users



Load Balancer



Compute (Web Server)



Database



Storage (Logs, Assets)

Look at this simple web application. What are the cost components?

- Compute (web server)
- Storage (application data, logs)
- Database (managed database)
- Network (data transfer to users)
- Request-based (e.g. API calls)

- Load balancer
- Backups and snapshots
- Monitoring and logging
- Security services
- Any other costs you can identify?



Junior Engineer Tip:

Always ask: What will this cost? Could it scale? Are there cheaper alternatives?

Understand costs early. Build smarter.



@veriqta



veriqta



@veriqta



@veriqta



@veriqta



@veriqta

VERIQT
ENGINEER TODAY.
A BRIGHTER TOMORROW.

Reading a Cloud Bill Without Getting Lost

UNDERSTAND YOUR BILL. INVESTIGATE COSTS. MAKE BETTER DECISIONS.

1

BILLING ACCOUNTS



- The top-level account that receives the bill.
- In AWS this is the management account.
- It can manage multiple accounts, users, and payment methods.

2

SERVICES



- Your bill is broken down by cloud services (e.g. EC2, S3, RDS).
- Each service has different pricing models.
- Identify which services are costing the most.

3

ACCOUNTS, SUBSCRIPTIONS AND PROJECTS



- Cloud providers allow multiple accounts or subscriptions.
- Used to separate environments, teams, or business units.
- Helps with cost allocation and ownership.

4

REGIONS



- Services are priced per region (e.g. us-east-1).
- Some regions are more expensive than others.
- Always check the region where the resource is running.

5

RESOURCE IDENTIFIERS



- Each resource has a unique identifier (e.g. instance ID, bucket name, ARN).
- Helps you find exactly which resource is incurring the cost.
- Useful when investigating unexpected charges.

6

USAGE QUANTITY



- The amount of resource used (e.g. hours, GB, requests).
- Measured differently per service.
- Higher usage means higher cost.

7

USAGE TYPE



- Describes what you used (e.g. BoxUsage, Requests, DataTransfer).
- Helps you understand how the cost was calculated.
- Each service has its own usage types.

8

RATE



- The price per unit of usage (e.g. \$0.023 per GB).
- Rates vary by service, region, and pricing model (on-demand, reserved, etc.).
- Check the unit (hour, GB, request) when comparing.

9

COST



- The total amount you pay for the usage.
- Calculated as: Usage quantity x Rate.
- This is what appears on your bill.

10

CREDITS



- Credits reduce your bill.
- Examples: free tier credits, promotional credits, migration credits.
- Shown as a negative amount on the bill.

11

DISCOUNTS



- Applied automatically (e.g. Reserved Instances, Savings Plans).
- Reduces the total cost.
- Check if your commitments are fully utilized to get the maximum discount.

12

TAXES



- Taxes are added based on your country and region.
- They vary by location and service.
- Always include taxes when forecasting total cost.

13

AMORTIZED vs ACTUAL COST



- Actual cost: what you are billed.
- Amortized cost: spreads the cost of commitments (e.g. Reserved Instances) over the term.
- Use amortized cost to understand the true cost of your resources.

14

WHY THE INVOICE TOTAL TELLS YOU VERY LITTLE



- The total bill hides which services, accounts, or teams are driving cost.
- You cannot see usage patterns, waste, or optimization opportunities.
- Engineers need to break down the bill to make informed decisions.

15

KEY TAKEAWAY



- Always dig deeper than the total amount.
- Break down the bill by service, account, region, and resource.
- Understand usage and pricing models.
- This helps you find waste and optimize costs.

15

EXAMPLE INVESTIGATION: FOLLOW THE MONEY



Cloud Bill
Start with your bill



Service
Which service is costing?



Account
Which account or subscription?



Region
Which region is it running in?



Resource
Which resource is consuming cost?



Usage
What is the usage type and quantity?

16

PRACTICE THIS



- 1 Open your cloud billing console
- 2 Find the top 3 services by cost
- 3 Drill down to the account and region
- 4 Identify the resources
- 5 Check the usage type and rate
- 6 Look for optimization opportunities



Junior Engineer Tip:

Treat your cloud bill like a log file. Investigate, don't guess.

Visibility today. Lower costs tomorrow.



@veriqta



veriqta



@veriqta



@veriqta



@veriqta



@veriqta

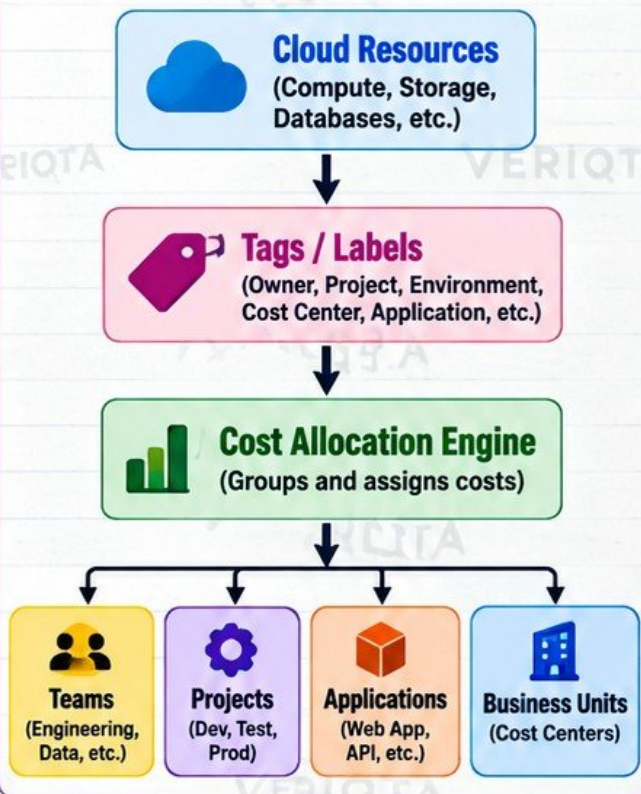
VERIQTA
ENGINEER TODAY.
A BRIGHTER TOMORROW.

Cost Allocation: Who Actually Spent the Money?

Why Organizations Allocate Cloud Costs

- Understand who is using cloud resources
- Increase accountability
- Enable cost optimization
- Support budgeting and forecasting
- Make fair and transparent decisions
- Identify unused or wasteful resources
- Align cloud spending with business value

How Cost Allocation Works



Beginner Mistake



Trying to optimize costs before determining who owns the resources.

If you do not know who owns the resources, you will:

- Optimize the wrong things
- Create conflicts between teams
- Miss the real cost drivers
- Struggle to get support for cost-saving actions.

Always identify ownership and allocation first, then optimize.

Key Concepts

Resource ownership	Identify the team or person responsible for the resource.
Accounts and subscriptions	Organize costs by cloud accounts (e.g. AWS accounts, Azure subscriptions).
Projects and environments	Separate costs by project names and environments (dev, test, prod).
Cost centers	Map costs to business units or departments.
Teams	Allocate costs to engineering, product, data, etc.
Applications	Track costs by application name.
Products	Group costs by product or service.
Shared infrastructure	Allocate shared costs like networking, security, and monitoring.

Tags and Labels

- ✓ Tags (AWS) or labels (GCP) add metadata to resources.
- ✓ Common tags: Owner, Project, Environment, CostCenter, Application, Team.
- ✓ Tags allow costs to be grouped and attributed.

Mandatory Tagging

- ✓ Set and enforce a tagging policy.
- ✓ Use automated tools (e.g. AWS Tag Policies, Azure Policy).
- ✓ Prevent untagged resources from being created.
- ✓ Regularly audit tagging compliance.

Untagged Resources

- ! Untagged resources are difficult to attribute.
- ! They appear as "unknown" in cost reports.
- ! Investigate and tag them as soon as possible.
- ! Use automation to detect and alert on untagged resources.

Showback vs Chargeback

Showback	Share cost information with teams. No direct billing. Used to create awareness and drive better decisions.
Chargeback	Directly bill teams or business units for their cloud usage. Used in mature organizations with strong cost governance.

Shared-Cost Allocation

- ✓ Some resources (e.g. networking, security, monitoring) are shared across teams.
- ✓ Use allocation rules (e.g. by usage, headcount, or percentage).
- ✓ Be transparent about how shared costs are distributed.
- ✓ Document the allocation method.

Tags, Labels, and the Metadata Behind FinOps

Tags and labels add metadata to cloud resources so you can track, organize and allocate costs.

What Are Cost-Allocation Tags?

- Key-value pairs added to cloud resources.
- Help identify who owns the resource, what it is used for, and how costs should be allocated.
- Used for cost allocation, reporting, automation and governance.
- Supported by all major cloud providers (AWS tags, Azure tags, GCP labels).



Example: Tagged Resource



environment = production
team = payments
application = checkout
cost-center = fin-001
project = online-store

Tagging Standards

- Define a mandatory set of tags for all resources.
- Use the same key names across the organization.
- Document allowed values (e.g. dev, test, prod).
- Apply tags during resource creation.
- Regularly audit for missing or incorrect tags.

Infrastructure as Code Tagging

Add tags in your IaC templates so tags are applied automatically.

```
# Terraform example (AWS)
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"
  tags = {
    environment = "production"
    team        = "payments"
    application  = "checkout"
    cost-center  = "fin-001"
    project      = "online-store"
  }
}
```

Common Tag Keys and Their Purpose

Tag Key	Purpose
team	Team that owns the resource.
application	Name of the application.
environment	Environment (dev, test, staging, prod).
owner	Person responsible for the resource.
cost-center	Business unit or department.
project	Project or initiative name.

Naming Conventions

- Use lowercase letters.
- Use hyphens instead of spaces (e.g. online-store).
- Keep names short and descriptive.
- Be consistent across all teams.
- Use a documented standard.
- Avoid special characters and uppercase letters.

Tag Enforcement

- Use organization policies (e.g. AWS Tag Policies, Azure Policy, GCP Org Policy).
- Prevent creation of resources without required tags.
- Use automated tools in CI/CD and Infrastructure as Code.
- Alert on non-compliant resources.
- Make tagging a requirement, not optional.

Tagging Kubernetes Workloads

Use labels (tags) on Kubernetes objects for cost allocation.

```
# Kubernetes Deployment with labels
apiVersion: apps/v1
kind: Deployment
metadata:
  name: checkout
  labels:
    environment: production
    team: payments
    application: checkout
spec:
  replicas: 3
  selector:
    matchLabels:
      app: checkout
```



These labels are used by cost tools like Kubecost to allocate Kubernetes costs.

Missing and Inconsistent Tags

- Resources without tags appear as "unknown" in cost reports.
- Inconsistent tag values (e.g. Prod vs production) break reporting.
- Tags added manually are often forgotten.
- It becomes harder to allocate and optimize costs.
- You may make the wrong cost decisions.

Beginner Mistake

- Trying to optimize costs before determining who owns the resources.

Why Tags Become Unreliable Without Governance

- Different teams use different tag names and values.
- Tags are missing, incorrect or outdated.
- No enforcement or automated checks.
- Manual tagging is inconsistent and error-prone.
- Without governance, cost allocation becomes inaccurate and trust in the data is lost.

Remember This

- Good tagging is the foundation of FinOps. You cannot manage what you cannot identify.

Budgets, Forecasts, and Cost Alerts

Plan your cloud spending, predict future costs, and get notified before costs become a problem.

What a Cloud Budget Does

- ✓ Sets an expected amount to spend.
- ✓ Helps you track spending against the plan.
- ✓ Provides visibility and control.
- ✓ Triggers alerts when costs are higher than expected.
- ✓ Helps teams make better decisions.
- ✓ Supports forecasting and cost governance.



Budget vs Spending Limit

Budget	Spending Limit
A planned amount for cost management.	A hard limit to stop spending (rarely used in cloud).
Helps you track and forecast.	May restrict or block usage (only in some services).
Used for visibility and alerts.	Not available for all cloud services.
Can be monthly, quarterly or yearly.	Usually set at the account or service level.

Types of Budgets



Monthly Budgets

Track expected spend each month.



Team Budgets

Allocate budgets per team.



Project Budgets

Track costs for specific projects or initiatives.



Application Budgets

Monitor spend per application or product.

Forecasted Spend

- ✓ Uses historical usage and current trends to predict future costs.
- ✓ Helps you see if you will exceed your budget.
- ✓ Actual spend is what you have spent so far.
- ✓ Compare actual vs forecast regularly.



Forecast helps you take action before you exceed your budget.

Cost Alerts



Threshold Alerts

Triggered when spend reaches a specific amount (e.g. \$1,000).



Percentage-Based Alerts

Triggered at a percentage of the budget (e.g. 50%, 80%, 100%).



Cost Anomaly Alerts

Detect unusual spikes in spending using machine learning.



Alert Recipients

Send alerts to the right people (e.g. team leads, finance, owners).

Why Alerts Need Owners

- ✓ Someone must review the alert and take action.
- ✓ Unowned alerts are ignored.
- ✓ Clear ownership ensures faster response.
- ✓ Link alerts to teams, projects or cost centers.

Avoiding Alert Fatigue

- ✓ Set meaningful thresholds (not too many alerts).
- ✓ Use the right alert types for your needs.
- ✓ Exclude expected spikes (e.g. month-end processes).
- ✓ Review and tune alerts regularly.
- ✓ Send alerts only to relevant people.

Failure Scenario



The cloud bill doubles, but nobody notices until the invoice arrives.

- No budgets or alerts were configured.
- Alerts were not assigned to anyone.
- Spending went unchecked for weeks.
- The organization faces an unexpected and high bill.
- Now the team must explain the overage and reduce costs quickly.

Remember This



Budgets, forecasts, and alerts give you time to act. They help you avoid surprises, control spending, and build a culture of cost awareness.

Investigating a Cloud Cost Spike

A step-by-step troubleshooting workflow to find and fix unexpected cloud costs.

Troubleshooting Workflow

- 1  **Unexpected Cost**
You notice costs are higher than expected.
- 2  **Confirm the Increase**
Check the billing console and verify the spike.
- 3  **Identify Service**
Find which service (e.g. EC2, S3, RDS) is driving the cost.
- 4  **Find Account / Region**
Check which account and region the cost is coming from.
- 5  **Inspect Usage**
Look at usage metrics, logs and billing details.
- 6  **Find Resource**
Identify the specific resource or group of resources.
- 7  **Determine What Changed**
Check for recent deployments, config changes, scaling or new workloads.
- 8  **Fix Cause**
Remove, right-size, stop or update the resource.
- 9  **Verify Cost Trend**
Monitor the cost to confirm it is back to normal.

Find the cause. Fix it. Keep monitoring.

What to Investigate

 New deployments	Recent releases may have created new resources.
 Scaling events	Auto-scaling may have increased resource count.
 Traffic increases	More users can increase compute, storage or data transfer.
 Forgotten resources	Unused resources (e.g. old instances, volumes, IPs).
 Storage growth	Logs, backups or data growth can increase storage costs.
 Logging explosions	Excessive logs can generate high ingestion and storage costs.
 Data transfer	High data transfer between regions or to the internet.
 Configuration changes	Instance type, storage type or settings may have changed.
 Compromised credentials	Attackers can create resources and increase costs.
 Unexpected workloads	Batch jobs, analytics or AI workloads may have run.

Useful Cloud Console Tools (Examples)

 Cost explorer	Analyze costs by service, account, region and time.
 Budgets and alerts	Check if alerts were triggered.
 Resource inventory	Find and review existing resources.
 Activity logs	Look for recent changes (e.g. CloudTrail, Activity Log).
 Monitoring metrics	Check usage metrics (e.g. CloudWatch, Azure Monitor).

Common Questions to Ask

- ? What changed recently?
- ? Which service is using the most?
- ? Which account and region is it in?
- ? Is this resource still needed?
- ? Was this expected (e.g. a new deployment)?
- ? Could this be an automated job or someone else's work?

Beginner Mistakes

- ✗ Ignoring small increases.
- ✗ Not checking all regions and accounts.
- ✗ Focusing on the wrong service.
- ✗ Assuming it is just normal growth.
- ✗ Not investigating until the invoice arrives.
- ✗ Deleting resources without understanding impact.

Pro Tip

Use tagging, alerts and regular reviews so you can detect cost issues early and avoid surprises.

Example: Cost Spike Investigation



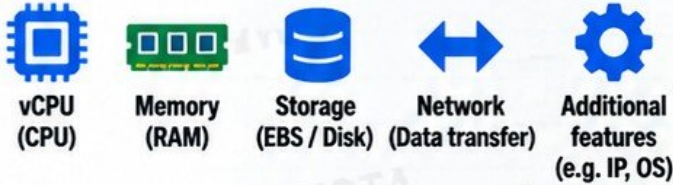
Spike detected!
Investigate the cause and take action.

Compute Costs, Stop Paying for Idle Machines

Understand compute pricing, rightsize your resources, and eliminate idle and wasted capacity.

Virtual Machine Cost Model

You pay for compute resources based on instance type, usage time and additional features.



Instance Families

Choose the right instance family for your workload.

Family	Optimized for	Example Use Cases
General purpose	Balanced CPU and memory	Web servers, dev env, small apps
Compute optimized	High CPU	Batch processing, media encoding
Memory optimized	High memory	Databases, caches (e.g. Redis)
Storage optimized	High disk throughput	Big data, logging, analytics
Accelerated compute	GPUs / specialized	Machine learning, AI workloads

Right Sizing: Match Resources to Real Usage

Use monitoring data to choose the right instance size.



Underutilization vs Overprovisioning

Underutilization

- Paying for more than you use.
- Common in dev and test environments.
- Examples: 5–10% CPU usage.

Overprovisioning

- Choosing a larger instance than needed.
- Increases costs without improving performance.
- Often happens “just in case”.

Performance vs Cost

Cheaper is not always better. Match the instance to your workload.

Lower Cost

- Smaller instances
- Spot instances
- Scheduled shutdowns



Better Performance

- Right instance type
- Enough CPU and memory
- Avoid under-provisioning

Goal: meet performance requirements at the lowest possible cost.

Real-World Scenario

A large VM runs at 5% CPU for an entire month.



Instance: 8 vCPU, 32 GB RAM
Average CPU usage: 5%
Monthly cost: \$500
Actual usage: very low

Result

\$500 spent for a machine that was mostly idle.

Action: Rightsize to a smaller instance or stop it when not needed.

Key Cost Optimization Strategies

	Rightsizing	Use the right instance size based on actual usage.
	Stop idle instances	Delete or stop instances you no longer need.
	Development environments	Use smaller instances. Shut down when not in use.
	Scheduling shutdowns	Automate start and stop times (e.g. dev/test).
	Autoscaling	Automatically increase or decrease capacity based on demand.
	Spot / preemptible capacity	Use unused cloud capacity at lower prices (for non-critical workloads).
	Reserved capacity	Commit to 1 or 3 years for lower discounts (e.g. Reserved Instances).
	Savings commitments	Commit to a certain amount of usage (e.g. AWS Savings Plans).

Common Places to Find Idle Compute

- ✓ Development and test environments
- ✓ Old or forgotten instances
- ✓ Instances left running after a project ends
- ✓ Unattached volumes (still incurring compute or storage costs)
- ✓ Auto-scaling groups with minimum capacity set too high

Beginner Mistakes

- ✗ Leaving dev/test environments running 24/7.
- ✗ Not checking actual CPU and memory usage.
- ✗ Choosing a large instance “just in case”.
- ✗ Ignoring idle instances and orphaned resources.
- ✗ Not using automation to stop non-production resources.

Remember This



Monitor usage, rightsize regularly, and automate shutdowns. Stop paying for idle machines.

Storage Costs, The Data You Forgot About

Store data intelligently. Understand storage costs, find waste, and use the right policies.

Types of Cloud Storage



Object Storage

- Stores unstructured data (files, backups, logs, media).
- Highly scalable and durable.
- Example: Amazon S3, Azure Blob Storage, Google Cloud Storage.



Block Storage

- Provides raw block storage for VMs.
- Used for operating systems and databases.
- Example: Amazon EBS, Azure Managed Disks, Google Persistent Disks.



File Storage

- Shared file system accessed by multiple servers.
- Used for applications that need a file system.
- Example: Amazon EFS, Azure Files, Google Filestore.

Storage Classes and Tiers

Use the right storage class based on how often you access the data.

Class / Tier	Use Case	Cost	Example (AWS)
Standard	Frequently accessed data	Higher	S3 Standard
Infrequent Access	Less frequently accessed data	Lower	S3 Standard-IA
Archive	Rarely accessed long-term data	Low	S3 Glacier
Deep Archive	Very long-term data (years)	Lowest	S3 Glacier Deep Archive

What You Are Charged For

Storage costs are more than just capacity.



Capacity charges

Cost per GB of data stored each month.



Request charges

Cost per API request (e.g. GET, PUT, LIST).



Retrieval charges

Cost to retrieve data from lower-cost tiers (e.g. archive).

Common Sources of Storage Cost Increases

- ✓ **Snapshots** Old snapshots that are no longer needed.
- ✓ **Backups** Retained backups taking up storage.
- ✓ **Old volumes** Unattached or forgotten disks.
- ✓ **Orphaned disks** Disks left behind after instances are deleted.
- ✓ **Log retention** Application and system logs stored for too long.
- ✓ **Development data** Test and dev environments storing large amounts of data.
- ✓ **Unstructured data** User uploads, media files, or database exports growing over time.

Lifecycle and Deletion Policies

- ✓ **Lifecycle policies** Automatically move data to cheaper tiers based on age or access patterns.
- ✓ **Deletion policies** Automatically delete data after a defined period.
- ✓ **Use retention policies** Set clear retention periods for logs, backups, and snapshots.
- ✓ **Review regularly** Audit and remove old or unnecessary data.

Why Cheap Storage Can Become Expensive

- ✓ Large amounts of data still add up, even at low cost per GB.
- ✓ Retrieval charges can be high when you need the data back.
- ✓ Many small files increase request costs.
- ✓ Logs, backups and snapshots often grow without notice.
- ✓ Data stored forever has a cost.
- ✓ Without governance, storage becomes a long-term financial risk.

Troubleshooting Scenario

Storage cost continues growing even though application traffic is stable.

Steps to Investigate

- 1 Check which storage service is increasing.
- 2 Look at storage usage trends over time.
- 3 Identify new or growing buckets, volumes, or file shares.
- 4 Check for unexpected snapshots or backups.
- 5 Review log retention and lifecycle policies.
- 6 Find orphaned or unattached disks.
- 7 Verify if any new data pipelines or exports are running.
- 8 Take action to delete, archive or apply lifecycle policies.
- 9 Monitor the cost trend after making changes.

Example: Storage Cost Breakdown (\$3)

Item	Usage	Cost (Example)
Standard storage	500 GB	\$11.50
Standard-IA storage	2 TB	\$25.00
Requests (GET/PUT)	1 million	\$5.00
Data retrieval (from Glacier)	500 GB	\$50.00
Total		\$91.50

Remember This



Storage is not free. Set lifecycle policies, monitor usage, and remove what you no longer need.

Network Costs, The Bill Engineers Often Miss

Data movement can be one of the biggest and most unexpected cloud costs.

Ingress vs Egress

Ingress (Inbound)



Data coming into your cloud environment from the internet.

- Usually low cost or free on most cloud providers.
- Examples: users accessing your app, file uploads.

Egress (Outbound)



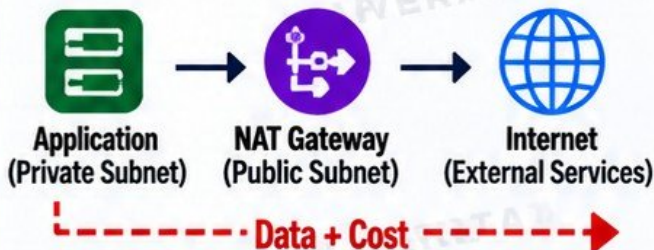
Data leaving your cloud environment to the internet or another region.

- Usually costs money (often significant).
- Examples: user downloads, API responses, backups to another region, software updates.

Common Network Cost Drivers

Internet egress	Data sent from your cloud to the internet. Often the largest network cost.
Cross-region traffic	Data transferred between different regions. Useful but expensive.
Cross-zone traffic	Data transferred between availability zones in the same region. Often overlooked.
NAT gateway costs	Used for private resources to access the internet. Charged per hour and per GB of data.
Load balancer charges	Application and network load balancers have hourly charges and data processing fees.
CDN costs	Content delivery networks reduce latency but still have data transfer costs.

Simple Architecture and Data Flow



When your application in a private subnet needs to access the internet (e.g. for updates, APIs), traffic goes through a NAT gateway. You pay for the NAT gateway hourly and for the data that leaves your cloud environment.

Other Sources of Network Costs

Private connectivity	Services like Direct Connect, VPN or Private Link have fixed and usage costs.
Architecture-driven costs	Your design (multi-region, multi-AZ, microservices) can increase network traffic and costs.
Kubernetes network traffic	East-west traffic between pods and services can add up, especially across nodes or zones.
Data replication	Replicating data for high availability and disaster recovery increases costs.
Third-party services	Sending data to external tools (monitoring, logging, backups) can generate egress costs.

Why Moving Data Can Cost More Than Storing It

- ✓ Storage is relatively cheap (e.g. \$0.02 per GB per month).
- ✓ Sending the same data over the internet can cost much more (e.g. \$0.09 per GB or higher).
- ✓ Frequent data transfers, backups, and cross-region traffic increase costs quickly.
- ✓ Many teams focus on compute and storage but ignore network costs.
- ✓ Optimizing data movement (e.g. using CDN, VPC endpoints, compression) can save significant money.

Real-World Example

A company stores 1 TB of data and sends it to the internet every month.

- Storage cost: \$0.02 per GB per month = \$20 per month
- Egress cost: \$0.09 per GB per month = \$90 per month
- Result: Sending the data costs 4.5 times more than storing it.

Remember This



Always consider network costs in your architecture. Data movement, not just storage, can make your cloud bill grow.

Database FinOps

Control Costs Without Compromising Performance or Availability

1. Managed Database Pricing



Managed databases (e.g. AWS RDS, Aurora, Azure SQL, Cloud SQL) are charged based on multiple components. Understanding these components helps you identify where the money is going and where you can optimize.

3. Common Cost Drivers



Idle databases

Databases running with little or no usage still incur cost.



Development databases

Often left running 24/7 and forgotten.



Overprovisioned instances

Larger instance sizes than needed.



Excess storage

Unused storage and high IOPS.



Long retention

Keeping backups and snapshots longer than required.



Unnecessary replicas

Read replicas that are not needed.

2. Key Cost Components



Instance size

Billed per hour based on CPU, memory and instance family.



Storage

Billed per GB per month.



IOPS

Additional cost for provisioned IOPS (e.g. io1, io2).



Backups

Automated backups and backup storage are charged separately.



Replicas

Read replicas incur additional cost.



Multi-AZ deployments

Higher cost because data is replicated to another AZ.



Serverless databases

Billed per second based on actual usage (ACU, vCPU, etc.).

4. Database Rightsizing



- ✓ Review CPU, memory and I/O utilization.
- ✓ Use monitoring data (e.g. CloudWatch, Azure Monitor).
- ✓ Rightsize to the smallest instance that meets performance requirements.
- ✓ Test changes in a non-production environment.

5. Reserved Database Capacity



- ✓ Use Reserved Instances or Savings Plans for predictable workloads.
- ✓ Can significantly reduce costs (up to 30-70%).
- ✓ Commit only for stable, long-term databases.
- ✓ Review regularly to avoid overcommitment.

6. Performance vs Cost



- Smaller instances cost less but may affect performance.
- Storage type and IOPS impact both cost and speed.
- Always validate performance before making changes.
- Use monitoring and load testing to understand the real impact.

7. Why Blindly Downsizing Is Dangerous



- Can cause high CPU or memory usage.
- May lead to slow queries and timeouts.
- Can impact application performance.
- May cause availability issues.
- Some databases cannot be easily scaled down without downtime.
- Always analyze usage and test before downsizing.

8. Production Awareness



Cost reduction must not create availability or performance incidents.

Always consider the business impact, test changes, monitor after changes, and have a rollback plan.

Optimize with data, not assumptions.

Kubernetes Cost Management

Understand, Control and Optimize Your Kubernetes Costs

1. Why Kubernetes Costs Are Difficult to Understand



- Many moving parts (clusters, nodes, pods, storage, networking).
- Resources are shared across teams and namespaces.
- Actual usage often differs from requested resources.
- Costs come from both cloud infrastructure and Kubernetes objects.
- It can be hard to see which workload is using the most resources.

2. Key Cost Components

Cluster costs	Control plane (managed or self-managed), region, and add-ons.
Nodes	Compute instances (e.g. EC2, VM). Your biggest cost component.
Pods	Run containers and consume CPU and memory on nodes.
CPU requests	Guarantee minimum CPU for a pod.
Memory requests	Guarantee minimum memory for a pod.
CPU limits	Maximum CPU a pod can use.
Memory limits	Maximum memory a pod can use.
Persistent volumes	Storage costs (e.g. EBS, EFS, PD).
Load balancers	Service type LoadBalancer incurs cloud provider costs.

3. Requested vs Actual Usage

Requested Resources

What you ask for (CPU and memory).

requests:
cpu: 500m
memory: 1Gi

- Used for scheduling.
- Reserves capacity on nodes.
- If requests are too high, you pay for unused capacity.

Actual Usage

What your application actually uses.

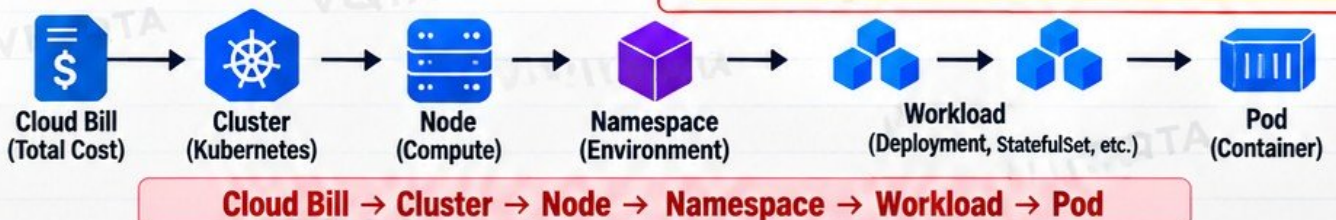
Usage:
cpu: 100m
memory: 300Mi

- Often lower than requests.
- Overprovisioning increases costs.
- Monitor usage and rightsize regularly.

4. Common Cost Drivers in Kubernetes

Overprovisioning	Requests and limits are higher than needed.
Idle capacity	Nodes or pods running with low usage.
Namespace allocation	Allocate costs by namespace.
Team allocation	Track costs by team.
Autoscaling	Scale pods based on demand.
Cluster autoscaler	Automatically adds or removes nodes.
Spot capacity	Use spot/preemptible instances for lower cost.
Development databases	Non-production workloads can be expensive if left running.

5. Key Mental Model



6. Cost per Workload



- Track costs at the workload or application level.
- Use labels and tags to identify workloads.
- Combine resource usage with cloud pricing.
- Helps you find high-cost workloads and optimize them.

7. Best Practices

- Set realistic CPU and memory requests and limits.
- Use monitoring to find unused or underutilized resources.
- Remove or stop idle development environments.
- Use autoscaling and spot capacity where appropriate.
- Allocate costs by namespace, team or application.
- Regularly review and rightsize your clusters.

8. Production Awareness



Cost optimization must not create availability or performance incidents. Always test changes, monitor impact and have a rollback plan.

Cheaper is not better if it breaks production.

Containers, Serverless, and Managed Services

Different Models. Different Costs. Make the Right Choice.

1. Container Cost Drivers



- Compute (CPU and memory requests and limits)
- Number of running containers
- Cluster nodes and instance types
- Storage (persistent volumes)
- Networking (load balancers, service mesh, data transfer)
- Idle and overprovisioned resources
- Orchestration platform costs (e.g. EKS, AKS, GKE)

Key Point:

You pay for the underlying compute nodes, whether the containers are fully used or not.

2. Serverless Pricing



- **Requests**
You are charged per invocation (e.g. per million requests).
- **Execution duration**
You pay for the time your code runs (in milliseconds).
- **Memory allocation**
Higher memory increases cost but also gives more CPU.
- **Provisioned concurrency**
Keep functions warm for low latency. You pay for the provisioned capacity even when idle.

Key Point:

You only pay when your code runs, but costs can increase quickly at scale.

3. Managed-Service Premiums



- You pay more for convenience and reduced operational work.
- **Examples:** Managed databases (RDS, Aurora), managed Kubernetes (EKS, AKS, GKE), managed cache (ElastiCache), managed messaging (SQS, Kafka), etc.
- Premium includes high availability, patching, backups, monitoring, and support.
- Reduces operational effort but usually costs more than self-managed options.

Key Point:

Managed services are more expensive, but they save time, reduce risk and operational overhead.

4. Operational Cost vs Infrastructure Cost



- **Infrastructure cost**
What you pay for compute, storage, networking, etc.
- **Operational cost**
The time, people and tools needed to manage, secure, monitor and keep the system running.

Remember:

A cheaper infrastructure option can become more expensive if it requires significant operational effort.

5. When Serverless Saves Money



- Variable or unpredictable workloads
- Infrequent or event-driven tasks
- Short-running functions
- No need to manage servers
- Ideal for APIs, background jobs, file processing, automation, etc.

6. When Serverless Becomes Expensive



- High and consistent traffic
- Long-running functions
- Large memory requirements
- Always-on workloads
- Heavy use of provisioned concurrency
- Better suited for containers or managed compute instead

7. Cost of Convenience



- You pay a premium for less operational work.
- Managed services, serverless and high-level abstractions are easier to use.
- Convenience reduces engineering time, but increases cost.
- The right choice depends on your team's time, skills and business priorities.

8. Comparing Architecture Options Properly

Factor	Containers (Self-Managed)	Serverless	Managed Services
Cost model	Pay for nodes (24/7)	Pay per request and execution time	Higher price per resource
Operational effort	High	Low	Low
Best for	Predictable, long-running workloads	Variable or event-driven workloads	Critical services, reduced ops work
Cost risk	Idle capacity	High usage can become expensive	Service premiums
Examples	EKS, AKS, GKE with EC2 nodes	AWS Lambda, Azure Functions, GCP Cloud Functions	RDS, Aurora, ElastiCache, SQS, etc.

9. Junior Engineer Tip



Cheapest infrastructure does not automatically mean lowest total operating cost.

Always consider performance, reliability, operational effort, team skills and business impact before choosing an architecture.

Discounts, Commitments, and Spot Capacity

Different pricing options exist to help you reduce cloud costs. The right choice depends on your workload type, business needs, and risk tolerance.

Pricing Options and How They Work

	On-demand pricing	Pay for compute resources by the hour or second. No long-term commitment.
	Reserved Instances (RIs)	Commit to a specific instance type and region for 1 or 3 years. Get significant discounts.
	Savings Plans	Commit to a consistent amount of usage (e.g. \$10/hour) for 1 or 3 years. More flexible than RIs.
	Committed-use discounts	Lower pricing in exchange for a commitment to use resources for a period of time.
	Spot instances	Use spare cloud capacity at much lower prices. Can be interrupted at any time.
	Preemptible capacity	Cloud provider can reclaim the resource with short notice. Very low cost, intended for fault-tolerant workloads.

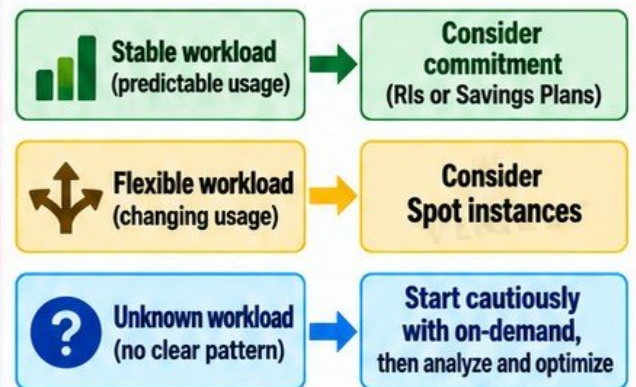
Key Concepts

Commitment utilization	The percentage of your committed resources or spend that you are actually using.
Commitment coverage	The percentage of your total usage that is covered by commitments (RIs or Savings Plans).
Break-even thinking	The point where the discount from a commitment becomes cheaper than on-demand pricing.
Workload predictability	How consistent and steady your resource usage is over time.
Interruptible workloads	Workloads that can handle being stopped or interrupted (e.g. batch jobs, data processing, CI/CD runners).
Risks of overcommitting	Paying for capacity you do not use, leading to wasted money.
Why discounts cannot repair waste	If you overprovision or run unused resources, you still waste money even with discounts.

When to Use Each Option

Pricing Option	Best For	Key Benefit	Key Risk
On-demand	Short-term or unpredictable workloads	No commitment, fully flexible	Highest cost per hour
Reserved Instances	Stable, long-term workloads	Large discounts (up to 72%)	Less flexible
Savings Plans	Steady usage across instance types/services	Significant discounts and flexibility	Commitment still required
Spot instances	Fault-tolerant, interruptible workloads	Very low cost (up to 90%)	Can be interrupted
Preemptible capacity	Non-critical, short-lived workloads	Lowest cost	Can be terminated at any time

Decision Model



Important Takeaways

- ✓ Choose the right pricing model based on workload predictability.
- ✓ Use commitment utilization and coverage to track effectiveness.
- ✓ Spot instances and preemptible capacity are best for non-critical and fault-tolerant workloads.
- ✓ Do not overcommit. Only commit when you are confident about usage.
- ✓ Discounts reduce costs but cannot fix waste from unused resources.

Beginner Mistake

✗ Trying to optimize costs before understanding your workloads and who owns the resources.

You might buy commitments or use Spot instances for the wrong resources, leading to wasted money or service interruptions. Always analyze usage, ownership, and workload characteristics first.

Unit Economics, Cost Per What?

Move beyond "our AWS bill is \$50,000." Understand the real cost per unit of value.

What Is Unit Economics?

Unit economics means understanding the cost to deliver a single unit of value for your business, such as a customer, transaction, API request, or deployment.



It helps you see the real cost efficiency of your cloud usage and connects engineering work to business value.

Key Terms

Unit cost	The cost to deliver one unit of value (e.g. one transaction or one customer).
Business metrics	Metrics that measure business value (e.g. customers, revenue, transactions).
Engineering metrics	Metrics that measure technical usage (e.g. requests, deployments, compute hours, storage).

Example Calculation

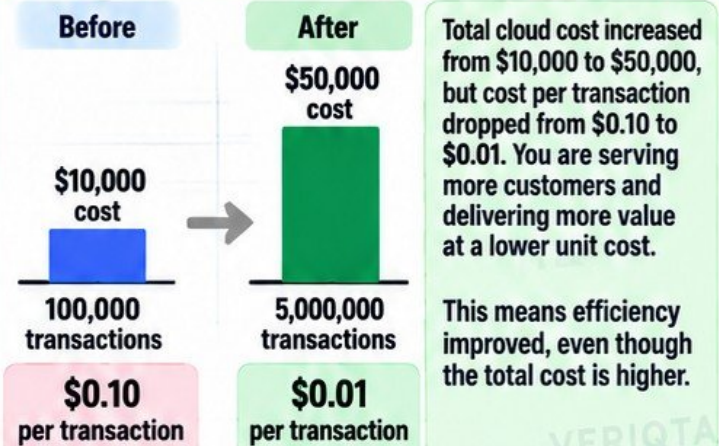
$$\begin{array}{c} \text{\$20,000} \\ \text{Total cloud cost} \\ \text{for a month} \end{array} \div \begin{array}{c} \text{2,000,000} \\ \text{Transactions} \\ \text{in the month} \end{array} = \text{\$0.01} \\ \text{per transaction}$$

$$\text{\$20,000 cloud cost} \div \text{2,000,000 transactions} = \text{\$0.01 per transaction}$$

Common Cloud Unit Costs

	Cost per customer	Total cloud cost ÷ number of customers.
	Cost per transaction	Total cloud cost ÷ number of transactions.
	Cost per API request	Total cloud cost ÷ number of API requests.
	Cost per deployment	Total cloud cost ÷ number of deployments.
	Cost per environment	Total cloud cost ÷ number of environments (dev, test, prod).
	Cost per workload	Total cloud cost ÷ number of workloads.
	Cost per tenant	Total cloud cost ÷ number of tenants (for multi-tenant apps).
	Cost per product	Total cloud cost ÷ number of products or services.

Why Total Cost Can Rise While Efficiency Improves



Important Takeaways

- ✓ Unit economics connects cloud costs to business value.
- ✓ Use both business and engineering metrics.
- ✓ Compare unit costs over time to measure efficiency.
- ✓ Helps with pricing, budgeting, and cost optimization.
- ✓ A higher total cost is not always bad if unit cost is lower.

Decision Questions

- 1 What is the key unit of value for your business?
- 2 What data do you need to calculate it?
- 3 How does your unit cost change over time?
- 4 What actions can reduce your unit cost?
- 5 Are you getting more value for your cloud spend?

FinOps Automation and Infrastructure as Code

Automate cost control. Prevent waste. Build it into your infrastructure from the start.

Why Manual Cost Management Does Not Scale

- ❌ Cloud environments change quickly.
- ❌ Many teams create resources every day.
- ❌ Manual tracking is slow and error prone.
- ❌ Unapproved resources and missed tags lead to wasted spend.
- ❌ It does not provide real-time visibility.
- ❌ It becomes impossible to manage at scale.



Automation makes cost management consistent, scalable, and proactive.

Terraform and Cost Awareness



- ✅ Define infrastructure using code.
- ✅ See what resources will be created.
- ✅ Estimate cost before deployment (e.g. using Infracost).
- ✅ Catch expensive changes early.
- ✅ Integrate cost checks into your workflow.

Preview infrastructure and cost estimate terraform plan

Use Infracost to see estimated cost changes infracost breakdown

Know the cost before you deploy.

Infrastructure as Code Policies



Tag enforcement

Require mandatory tags (e.g. Owner, Environment, CostCenter).



Policy as Code

Use tools like OPA, Sentinel or AWS Config rules to enforce policies.



Resource policies

Restrict creation of expensive or non-compliant resources.



Prevent costly infrastructure

Block non-compliant or high-cost resources before deployment.

Automation for Cost Control



Budget automation

Set up budget alerts and automated actions.



Automated shutdowns

Automatically stop non-production resources.



Lifecycle automation

Start, stop, or delete resources based on schedules or rules.



Detecting idle resources

Find and remove unused or underutilized resources.

DevOps Connection



Pull Request



IaC Change



Cost Check



Review



Deploy

Catch and prevent unnecessary costs before they reach production.

Key Takeaways

- ✅ Automate cost management. Manual processes do not scale.
- ✅ Use Terraform and cost tools to see costs early.
- ✅ Enforce tags and policies with automation.
- ✅ Detect and remove idle resources.
- ✅ Integrate cost checks into your CI/CD pipeline.

CI/CD Cost Checks



Pull-request cost visibility

Show estimated cost changes in the pull request.



Guardrails vs manual approval

Use automated guardrails instead of relying only on human approval.

Remember



- Cost control starts before deployment.
- Automation reduces human error.
- Policies and tagging keep your environment organized.
- Visibility in pull requests helps better decisions.
- The goal is not just to reduce cost, but to prevent waste and build sustainable cloud operations.



The best time to stop an expensive resource is before it is created.

FinOps Observability, Metrics for Money

Use data, not guesswork, to understand, explain, and optimize cloud costs.

What Is FinOps Observability?

FinOps observability means using data and metrics to understand how cloud resources are used, what they cost, and where you can improve. It brings together cost data, usage data, and operational telemetry so you can see the full picture.



The same observability principles we use for system performance also apply to cost.

Connecting Telemetry with Cost

Operational Telemetry



Cloud Cost Data



Unified View



- Cost-aware dashboards
- Correlate cost with usage
- Find waste
- Make better decisions

Combine operational and cost data to understand why costs change, not just that they changed.

Key Metrics for Money

	Cost dashboards	Visualize total and breakdown of cloud costs.
	Cost trends	See how costs change over time.
	Usage trends	Track usage growth or reduction.
	Utilization metrics	Measure how efficiently resources are used.
	Cost anomalies	Detect unexpected spikes or drops.
	Forecast variance	Compare forecasted vs actual costs.
	Budget variance	Track budget vs actual spend.
	Commitment utilization	See how much of your RIs or Savings Plans you use.
	Commitment coverage	Measure what percentage of usage is covered.
	Idle-resource metrics	Find underutilized or idle resources.
	Cost allocation coverage	Check how much of your costs are properly tagged and allocated.
	Cost per unit	Understand cost per customer, transaction, API request, etc.

Key Idea



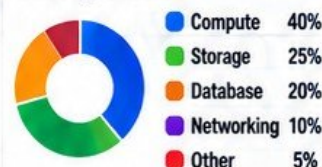
Engineers should investigate cost changes with the same evidence-driven mindset used for production incidents.

Cost-Aware Dashboards

Monthly Cloud Cost



Cost by Service



Usage vs Cost Trend



Alerts / Anomalies

- ! Cost increased by 35%
- ! Unusual storage growth
- ! Idle resources detected
- ! Forecast exceeds budget

Practical Tips

- ✓ Build dashboards for your team and stakeholders.
- ✓ Set up anomaly alerts.
- ✓ Track both cost and usage metrics.
- ✓ Use tags to improve cost allocation.
- ✓ Regularly review commitment utilization and coverage.
- ✓ Investigate cost changes using real data (metrics, logs, usage).

Cost Optimization Without Breaking Production

Reduce cloud costs while keeping your systems reliable, secure, and high performing.

Waste vs Necessary Redundancy

Waste

Unnecessary resources that do not add business value.

- ✗ Unused resources
- ✗ Oversized instances
- ✗ Idle storage
- ✗ Unattached volumes
- ✗ Forgotten test environments
- ✗ Unnecessary data transfer
- ✗ Unmanaged licenses

Necessary Redundancy

Extra capacity that protects availability, performance, and the business.

- ✓ High availability
- ✓ Disaster recovery
- ✓ Backups
- ✓ Capacity headroom
- ✓ Security controls
- ✓ Compliance requirements
- ✓ SLOs



Not all unused-looking resources are waste. Understand the purpose before removing them.

Why Reliability Has a Cost

Reliable systems require additional resources, and these resources increase costs. They exist to prevent downtime, data loss, and service degradation.

You cannot optimize costs without understanding what keeps your business running.



High availability
Runs multiple instances across zones or regions.



Disaster recovery
Keeps a full or partial copy in another region.



Backups
Stores previous versions of your data.



Capacity headroom
Handles traffic spikes.



Security controls
Require additional tools and resources.



Compliance requirements
May require data retention, encryption, and logging.



SLOs (Service Level Objectives)
Require investment to meet availability and performance targets.

Safe Rightsizing Process

- 1 **Analyze usage** Review metrics (CPU, memory, network, storage) over time.
- 2 **Identify candidates** Find resources that are oversized or underutilized.
- 3 **Test before changing** Try a smaller size in a non-production environment or during a low-traffic period.
- 4 **Monitor performance** Check for errors, latency, or increased resource usage.
- 5 **Implement the change** Resize, remove, or adjust capacity.
- 6 **Verify savings** Confirm lower costs in your billing data.

Change Management and Rollback

- | | |
|------------------------------|---|
| Plan the change | Document what you are changing and why. |
| Communicate | Inform stakeholders and teams. |
| Execute in stages | Make small, controlled changes. |
| Monitor closely | Watch metrics, logs, and alerts for issues. |
| Be ready to roll back | Have a clear rollback plan if problems occur. |

Critical Warning



Never delete or downsize infrastructure solely because it "looks expensive."

A resource may look unused but could be critical for high availability, disaster recovery, security, compliance, or future growth. Always understand the purpose, validate the impact, and test before making changes.

Common Optimization Opportunities

- | | |
|-----------------------------|---|
| Rightsize compute | Use the right instance size for actual usage. |
| Optimize storage | Use the right storage class and remove unused data. |
| Reduce data transfer | Minimize unnecessary data movement. |
| Clean up resources | Remove unused or abandoned resources. |
| Review licenses | Eliminate unused or over-provisioned licenses. |
| Use automation | Schedule shutdowns for non-production environments. |

Key Takeaways

- ✓ Optimize costs without compromising reliability.
- ✓ Understand the difference between waste and necessary redundancy.
- ✓ Always test and monitor before downsizing.
- ✓ Use change management and have a rollback plan.
- ✓ Verify savings after changes.
- ✓ Cost optimization is an ongoing process, not a one-time activity.

FinOps Incident Lab, The Cloud Bill Exploded

A realistic junior engineer investigation.

The Scenario



Cloud spending increases 65% in seven days.

The Evidence



Compute cost increased



Network egress increased



Kubernetes node count increased



Logging volume increased

The Learner Investigates

- 1 What changed?
- 2 Which service created the increase?
- 3 Which team owns it?
- 4 Was traffic legitimate?
- 5 Did autoscaling behave correctly?
- 6 Were resource requests changed?
- 7 Did log verbosity change?
- 8 Is abnormal traffic involved?



What Should Be Fixed?

- ✓ Stop or scale down unused resources.
- ✓ Fix misconfigured autoscaling.
- ✓ Reduce unnecessary logging.
- ✓ Optimize network usage.
- ✓ Right-size compute and storage.



What Should Not Be Changed?

- ✗ Do not reduce capacity for critical workloads.
- ✗ Do not disable monitoring or logging.
- ✗ Do not remove security controls.
- ✗ Do not make changes without understanding the root cause.



How Will the Engineer Verify the Result?

- ✓ Monitor costs for the next few days.
- ✓ Confirm the increase has stopped.
- ✓ Check key metrics and dashboards.
- ✓ Validate the service is healthy.
- ✓ Review logs and alerts.



The Troubleshooting Model



Junior Engineer Tip

Treat cost incidents like production incidents.
Use data, ask the right questions, and fix the real cause, not just the symptoms.

The Junior Engineer's FinOps Production Checklist

Plan. Monitor. Optimize. Verify. Keep costs under control without breaking production.

1. Before Creating Infrastructure

Ask these questions before you deploy anything.

- ✓ Who owns it?
- ✓ Is it tagged?
- ✓ What will it cost?
- ✓ Does it need to run continuously?
- ✓ What scaling behavior is expected?



**PLAN
BEFORE
YOU DEPLOY**

2. While It Is Running

Monitor your resources and costs regularly.

- ✓ Is utilization reasonable?
- ✓ Is spending within expectations?
- ✓ Are anomalies appearing?
- ✓ Are logs or storage growing?
- ✓ Are resources idle?



**OBSERVE
AND
DETECT EARLY**

3. Before Optimizing

Understand the impact before making changes.

- ✓ Understand the workload.
- ✓ Check performance.
- ✓ Check dependencies.
- ✓ Understand reliability requirements.
- ✓ Estimate impact.
- ✓ Create a rollback plan.



**ANALYZE
BEFORE
YOU CHANGE**

4. After Optimizing

Verify the results and keep it stable.

- ✓ Verify application health.
- ✓ Verify performance.
- ✓ Verify reliability.
- ✓ Verify actual savings.
- ✓ Document the change.



**CONFIRM
AND
IMPROVE**

Critical Reminder



Cost optimization should never sacrifice reliability, security, compliance, or user experience.

Junior Engineer Mindset



- Be curious about cost changes.
- Use data, not assumptions.
- Make small, safe changes.
- Always verify the outcome.
- FinOps is an ongoing process, not a one-time task.



Goal: Build reliable, cost-efficient, and sustainable systems.
Good engineers deploy infrastructure. Great engineers run it efficiently.